

AnzeigePanel

```
import java.awt.*;
import javax.swing.*;
import java.util.concurrent.ThreadLocalRandom;

public class AnzeigePanel extends JPanel
{
    private final int minZahl = 1;
    private final int maxZahl = 100;
    private int gesuchteZahl = 0 ;
    private int versuche = 0;
    private JLabel labelSpieltext = new JLabel("Rate meine Nummer! (1-100)");
    private JLabel labelName = new JLabel("Name:");
    private JLabel labelVersuche = new JLabel("Versuche");
    private JLabel labelScore = new JLabel("Highscore");
    private JLabel labelErgebnis = new JLabel("-");
    private JTextField tfName = new JTextField("Name eingeben",10);
    private JTextField tfVersuche = new JTextField("0",2);
    private JTextField tfHighscore = new JTextField("-",2);
    private JTextField tfEingabe = new JTextField("0",2);

    private DateiManager datManager;

    public AnzeigePanel(DateiManager datManager) {
        this.datManager = datManager;
        setPreferredSize(new Dimension(100,100));

        setLayout(new FlowLayout(FlowLayout.LEFT,10,20));

        tfHighscore.setEditable(false);
        tfVersuche.setEditable(false);

        add(labelSpieltext);
        add(labelName);
        add(tfName);

        add(labelVersuche);
        add(tfVersuche);
        add(labelScore);
        add(tfHighscore);
        add(tfEingabe);
        add(labelErgebnis);
    }
}
```

Wenden Sie sich nun der Klasse `AnzeigePanel` zu. Hier sollen zunächst folgende zwei Methoden implementiert werden.

Die erste Methode

```
public void starteSpiel()
```

soll folgenden Funktionsumfang haben:

- Die Variable `versuche` wird auf 0 gesetzt und dieses wird im entsprechenden Textfeld angezeigt
- Das Label für das Ergebnis soll auf „-“ gesetzt und das Textfeld für die Eingabe auf 0 gesetzt werden.
- Das Textfeld für den Spielernamen wird auf nicht editierbar gesetzt
- Die Methode `leseListeHighscore()` der Klasse `DateiManager` wird aufgerufen
- Der eingegebene Spielername wird ausgelesen und an die Methode `String getPunkteHighscoreVonName(String name)` der Klasse `DateiManager` übergeben. Der Rückgabewert wird im entsprechenden Textfeld für den Highscore angezeigt.
- Die Methode `berechneNeueZufallszahl()` wird aufgerufen.

Die zweite Methode

```
public void aufgeben ()
```

soll folgenden Funktionsumfang haben:

- Das Label für das Ergebnis soll den String „Die gesuchte Zahl war “ anzeigen und den Wert der Variablen `gesuchteZahl` am Ende hinzufügen.
- Das Textfeld für den Spielernamen soll auf editierbar gesetzt werden.

```
public void starteSpiel() {
    versuche=0;
    tfVersuche.setText(""+versuche);
    labelErgebnis.setText("-");
    tfEingabe.setText("0");
    tfName.setEditable(false);
    datManager leseListeHighscore();

    tfHighscore.setText(datManager.getPunkteHighscoreVonName(tfName.getText()))
;
    berechneNeueZufallszahl();
}
```

In der Klasse `AnzeigePanel` fehlt nun noch die letzte Methode

```
public boolean ueberpruefeZahl()
{
    public boolean ueberpruefeZahl() {
        int eingegebeneZahl =
konvertiereStringZahlZuInteger(tfEingabe.getText());
        boolean richtigGeraten = eingegebeneZahl==gesuchteZahl;
        if(richtigGeraten)
        {
            labelErgebnis.setText("Richtig geraten!");
            versuche++;
            tfVersuche.setText(""+versuche);
            tfName.setEditable(true);
            datManager.neuerEintragListeHighscore(tfName.getText(),
tfVersuche.getText());
            datManager.schreibeListeHighscore();
        }
        else {
            versuche++;
            tfVersuche.setText(""+versuche);
            if(eingegebeneZahl>gesuchteZahl) {
                labelErgebnis.setText("Kleiner!");
            }
            else{
```

```

        labelErgebnis.setText("Größer!");
    }
    }
    return richtigGeraten;
}

public void aufgeben() {
    labelErgebnis.setText("Die gesuchte Zahl war " + gesuchteZahl);
    tfName.setEditable(true);
}

/*
 * Hilfsmethode berechnet eine neue Zufallszahl und speichert diese in
der Klassenvariable 'gesuchteZahl'
 */
private void berechneNeueZufallszahl() {
    gesuchteZahl = ThreadLocalRandom.current().nextInt(minZahl, maxZahl
+ 1);
    //System.out.println(gesuchteZahl); //Auskommentieren um direkt
beim Start das Ergebnis zu kennen.
}

/*
 * Hilfsmethode konvertiert einen String in einen Integer.
 * Bedingung ist, dass der String ausschließlich zahlen enthaehlt.
 */
private int konvertiereStringZahlZuInteger(String zahl) {
    int eingegebeneZahl=0;
    try{
        eingegebeneZahl = Integer.parseInt(zahl);
    }
    catch (NumberFormatException e)
    {
        System.out.println("Nur zahlen eingeben!");
    }
    return eingegebeneZahl;
}
}

```

ButtonPanel

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ButtonPanel extends JPanel
{
    private AnzeigePanel anzeigePanel ;
    private JButton buttonStart = new JButton("START");
    private JButton buttonAufgeben = new JButton("AUFGEBEN");
    private JButton buttonRaten = new JButton("RATEN");

    public ButtonPanel (AnzeigePanel anzeigePanel) {
        this.anzeigePanel = anzeigePanel;
        setLayout (new FlowLayout (FlowLayout.CENTER, 30, 20));
        add(buttonStart);

        //Create UI
    }
}

```

```

        add(buttonRaten);
        add(buttonAufgeben);
        //Listeners
        buttonStart.addActionListener(new ButtonListener());
        buttonAufgeben.addActionListener(new ButtonListener());
        buttonRaten.addActionListener(new ButtonListener());

        //Init buttons
        buttonAufgeben.setEnabled(false);
        buttonRaten.setEnabled(false);
    }

```

Zur Vervollständigung der Funktion Ihrer Oberfläche soll nun die innere Klasse `ButtonListener` der Klasse `ButtonPanel` implementiert werden.

Bei einem Klick auf den Button „Start“ soll folgendes passieren:

- Der Button „Start“ wird deaktiviert
- Der Button „Aufgeben“ wird aktiviert
- Der Button „Raten“ wird aktiviert
- Die Methode `starteSpiel()` der Klasse `AnzeigePanel` wird aufgerufen

```

class ButtonListener implements ActionListener
{
    public void actionPerformed(ActionEvent e) {
        if(e.getSource()==buttonStart) {
            buttonStart.setEnabled(false);
            buttonAufgeben.setEnabled(true);
            buttonRaten.setEnabled(true);
            anzeigePanel.starteSpiel();
        }
        if(e.getSource()==buttonAufgeben) {
            buttonStart.setEnabled(true);
            buttonAufgeben.setEnabled(false);
            buttonRaten.setEnabled(false);
            anzeigePanel.aufgeben();
        }
        if(e.getSource()==buttonRaten) {
            if(anzeigePanel.ueberpruefeZahl()) {
                buttonStart.setEnabled(true);
                buttonAufgeben.setEnabled(false);
                buttonRaten.setEnabled(false);
            }
        }
    }
}

```

DateiManager

```

import java.io.*;
import java.util.*;

public class DateiManager
{
    private final String pfadZurHighscoreDatei =

```

```

"..\resources\\Highscores.txt";
    private FileReader fr;
    private BufferedReader br;
    private FileWriter fw;
    private BufferedWriter bw;

    ArrayList<Highscore> listeAllerHighscores = new ArrayList<>();

```

Nachdem die Oberfläche von Ihnen erstellt wurde, soll nun in der Klasse `DateiManager` die Methode

```
public void leseListeHighscore()
```

implementiert werden. In dieser Methode sollen alle abgespeicherten Spielernamen/Punkte-Kombinationen aus der Datei *Highscores.txt* eingelesen werden. Der Pfad der Datei ist in der Klassenvariable `pfadZurHighscoreDatei` gespeichert.

Lesen Sie dazu immer zwei Zeilen (entspricht Spielernamen und Punktestand) der Datei ein und erzeugen mit diesen beiden eingelesenen Zeilen ein Objekt der Klasse `Highscore`. Dazu steht Ihnen der Konstruktor der Klasse `Highscore`

```
public Highscore(String name, String punkte)
```

zur Verfügung.

Das erzeugte Objekt wird anschließend der Liste vom Typ `ArrayList<Highscore>` hinzugefügt. Die Liste wird so lange erweitert, bis alle Spielernamen/Punkte-Kombinationen aus der Textdatei eingelesen wurden.

```

public void leseListeHighscore() {
    listeAllerHighscores.clear();

    try {
        fr = new FileReader(pfadZurHighscoreDatei);
        br = new BufferedReader(fr);
        String name;
        String punkte;
        while((name = br.readLine()) != null)
        {
            punkte = br.readLine();
            listeAllerHighscores.add(new Highscore(name, punkte));
        }
        br.close();
    }
    catch(FileNotFoundException e) {
        System.out.println("Datei nicht gefunden!");
    }
    catch(IOException e) {
        System.out.println("Fehler!");
    }
}

```

In der Klasse `DateiManager` befinden sich zwei weitere Methoden, welche von Ihnen implementiert werden müssen.

Mit Hilfe der Methode

```
public void schreibeListeHighscore()
```

```

public void schreibeListeHighscore() {
    try {
        fw = new FileWriter(pfadZurHighscoreDatei);
        bw = new BufferedWriter(fw);

        for(Highscore highscore: listeAllerHighscores) {
            bw.write(highscore.getName());
            bw.newLine();
            bw.write(highscore.getPunkte());

```

```

        bw.newLine();
    }
    bw.close();
}
catch(IOException e) {
    System.out.println("Fehler!");
}
}

```

Die zweite zu implementierende Methode

```
public void neuerEintragListeHighscore(String name, String punkte)
```

soll ein neues Objekt vom Typ `Highscore` erzeugen und der Liste aller Highscores hinzufügen.

```

public void neuerEintragListeHighscore(String name, String punkte) {
    listeAllerHighscores.add(new Highscore(name, punkte));
}

public String getPunkteHighscoreVonName(String name) {
    Optional<Highscore> highscoreVonName
=listeAllerHighscores.stream().filter(highscore ->
highscore.getName().equals(name))
    .min(Comparator.comparing(high->
Integer.parseInt(high.getPunkte())));
    if(highscoreVonName.isPresent()) {
        return highscoreVonName.get().getPunkte();
    }else {
        return "-";
    }
}
}

```

GrafikPanel

```

import javax.swing.*;
import java.awt.*;
import javax.imageio.*;
import java.awt.image.*;
import java.io.*;
import java.awt.geom.*;

public class GrafikPanel extends JPanel
{
    private BufferedImage image=null;
    private AffineTransform scaleTransfo =
AffineTransform.getScaleInstance(0.1,0.1);
    private String pathToImage=".\\resources\\Fragezeichen.png";
    public GrafikPanel() {
        try
        {
            image = ImageIO.read(new File(pathToImage));
        }
        catch (IOException e)
        {
            System.out.println("Images could not be read");
        }
        setPreferredSize(new Dimension(150,400));
        repaint();
    }
}

```

```

    }

    /**
     * Methode zum Zeichnen des Bildes
     */
    public void paint(Graphics g)
    {
        Graphics2D g2 = (Graphics2D) g;
        super.paint(g2);

        g2.transform(scaleTransfo);
        g2.drawImage(image, 0, 0, this);
    }
}

```

Gui

```

import javax.swing.*.*;
import java.awt.*.*;

public class Gui extends JFrame
{
    private GrafikPanel grafikPanel = new GrafikPanel();
    private DateiManager datManager = new DateiManager();
    private AnzeigePanel anzeigePanel = new AnzeigePanel(datManager);
    private ButtonPanel buttonPanel = new ButtonPanel(anzeigePanel);

    public Gui() {
        super("Zahlen raten");
        setLayout(new BorderLayout());
        add(grafikPanel, BorderLayout.WEST);
        add(anzeigePanel, BorderLayout.CENTER);
        add(buttonPanel, BorderLayout.SOUTH);
        setSize(375, 260);
        setVisible(true);
    }

    public static void main(String [] args) {
        new Gui();
    }
}

```